

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0606 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K-Nearest Neighbour Learning – Locally Weighted Regression – Radial Basis Functions – Case Based Learning (BL 5)

Assessment Tool Weightage: 10%

QUESTIONS: 05 Total Marks: 100

Annexure A — Comparative Analysis

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

1. Compare K-Nearest Neighbour and Case-Based Learning using a real-world example such as medical diagnosis. Analyze differences in memory usage, similarity measures, and prediction strategy.

Overview: K-Nearest Neighbour (KNN) and Case-Based Learning (CBL) are both instance-based (lazy) learners that defer generalization until a query is made, but they differ substantially in how they represent and reuse stored experience. Consider a medical diagnosis system that predicts whether a patient has a particular disease based on past patient records.

Memory Usage: KNN stores the entire training set as raw, fixed-length feature vectors (e.g., age, blood pressure, glucose level) with no additional structure beyond the data points themselves. Memory grows linearly with the number of patients and every attribute is retained in the same numeric/categorical form. Case-Based Learning, in contrast, stores richer "cases" — a case is a structured record that may include the patient's symptoms, treatment history, outcome, and even textual notes or exceptions. CBL memory therefore holds more semantically rich, often variable-length representations, and it typically also retains meta-knowledge (indexes, adaptation rules) used to organize and retrieve cases efficiently, so its memory footprint per instance is usually larger than KNN's simple vector storage.

Similarity Measures: KNN relies on strict mathematical distance metrics such as Euclidean, Manhattan, or Minkowski distance computed directly over numeric feature vectors; categorical attributes need to be encoded numerically beforehand. CBL uses more flexible, often domain-specific similarity functions that can weigh symptoms differently depending on medical relevance, handle missing or qualitative attributes, and incorporate expert-defined rules (e.g., "a persistent cough matters more than a mild headache"). This makes CBL's similarity assessment closer to how a clinician reasons by analogy, whereas KNN's is a purely geometric computation.

Prediction Strategy: KNN predicts by taking a simple majority vote (classification) or average (regression) of the K nearest neighbours' labels, with no adaptation of the retrieved cases. CBL follows a four-step reasoning cycle — Retrieve the most similar past case(s), Reuse the stored solution, Revise it if it does not perfectly fit the new patient, and Retain the new case (with outcome) for future reference. This means CBL can adapt a previous treatment plan to the specifics of a new patient, while KNN only aggregates labels without any explanation or adaptation.

Conclusion: In short, KNN is a purely statistical, low-overhead nearest-neighbour voter well suited to numeric diagnostic data, while CBL is a knowledge-rich, explanation-capable framework that mirrors clinical reasoning by adapting past treatment cases, at the cost of higher memory and design complexity.

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

Overview: Locally Weighted Regression (LWR) and Radial Basis Function (RBF) networks are both used to approximate continuous, non-linear functions, such as predicting house price from square footage and location in a real-estate dataset. Both exploit locality, but they implement it very differently.

Locality: LWR is a purely lazy, memory-based method: for every new query point, it fits a fresh local (usually linear or low-order polynomial) regression model using only training points close to that query, weighted by a kernel (typically Gaussian) that decays with distance. Locality is therefore query-specific and recomputed every time. RBF networks instead fix a set of basis-function centres (chosen via clustering such as k-means, or randomly) during training; each hidden neuron represents a fixed local

region of the input space, and this structure is trained once and reused for all future queries. Locality in RBF is therefore baked into the trained model rather than recomputed per query.

Computational Cost: LWR has cheap "training" (nothing is precomputed) but expensive prediction — every query requires searching for neighbours and solving a weighted least-squares problem, making it slow for large datasets or many queries (e.g., predicting prices for thousands of houses repeatedly). RBF networks invert this trade-off: training is more expensive because centres, widths, and output-layer weights must be learned (often via k-means plus linear regression or gradient descent), but prediction is fast — a new query simply passes through the fixed network in constant time regardless of training-set size. For applications needing many repeated predictions (e.g., a live pricing API), RBF is more efficient; for one-off or rarely-changing queries, LWR is simpler.

Smoothness of Predictions: Because each query in LWR fits its own local model, the transition between neighbouring query regions can be less smooth if the kernel bandwidth is poorly chosen, sometimes producing a wiggly response surface. RBF networks generally yield smoother global approximations because the basis functions (typically Gaussians) overlap continuously across the input space and the output is a fixed weighted sum of these smooth functions, giving a globally consistent and differentiable surface.

Conclusion: LWR trades prediction-time cost for adaptivity with no training phase, while RBF networks trade upfront training cost for fast, smooth, reusable predictions — making RBF preferable for large-scale, repeated-query settings like the house-price example, and LWR preferable for small, dynamic datasets that change frequently.

3. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.

Overview: For spam detection, an email is represented as a high-dimensional vector of word/feature counts (bag-of-words). Naïve Bayes (NB) and Logistic Regression (LR) are the two classical baselines for this task, one generative and one discriminative.

Feature Independence Assumption: Naïve Bayes assumes conditional independence of features given the class — i.e., it assumes the presence of the word "free" is independent of the presence of "win" given that the email is spam. This assumption is almost always violated in real text (words co-occur), yet NB often still performs well in practice. Logistic Regression makes no such independence assumption; it directly models the decision boundary as a weighted linear combination of all features, allowing correlated features to jointly and additively influence the outcome without a strong distributional assumption.

Probability Estimation: NB is a generative model: it estimates the class-conditional feature likelihoods $P(\text{word}|\text{class})$ and the class prior $P(\text{class})$ from training data, then applies Bayes' theorem to compute the posterior $P(\text{class}|\text{email})$. LR is discriminative: it directly models the posterior $P(\text{class}|\text{features})$ using the logistic (sigmoid) function applied to a linear combination of features, learned by maximizing conditional likelihood (typically via gradient descent). Because LR optimizes the quantity it is actually evaluated on, its probability estimates tend to be better calibrated when the independence assumption in NB is strongly violated.

Computational Efficiency: NB is extremely fast to train — it only requires counting word frequencies per class (closed-form estimation), making it well suited to very large email corpora and online/incremental updates. LR requires an iterative optimization procedure (gradient descent or Newton's method) to fit its

weights, which is more computationally expensive, especially as the vocabulary size grows, though still tractable for typical spam-filtering vocabularies.

Performance on High-Dimensional Data: Text data is inherently high-dimensional and sparse (large vocabularies, most words absent in a given email). NB tends to perform well here despite its naive assumption because it needs relatively little data per feature to estimate simple frequency counts, making it robust with limited training data. LR can perform better when there is sufficient training data to reliably estimate feature weights and when features are correlated, but it is more prone to overfitting in very high dimensions without regularization (L1/L2).

Scenarios of Superiority: Naïve Bayes tends to outperform when training data is scarce, the vocabulary is very large, or fast/incremental retraining is needed (e.g., a lightweight spam filter on a mail server). Logistic Regression tends to outperform when ample labelled data is available, feature correlations are important, and well-calibrated probabilities are needed for downstream decisions (e.g., ranking emails by spam probability with a tunable threshold).

4. Analyze the differences between Support Vector Machines (SVM) and KNN in classification tasks. Use an example dataset to compare decision boundaries, scalability, sensitivity to noise, and computational complexity. Discuss how kernel functions in SVM improve performance for non-linear problems.

Overview: Consider a dataset of handwritten digit images (e.g., distinguishing '3' from '8') represented as pixel-intensity feature vectors. SVM and KNN represent two fundamentally different classification paradigms: SVM is an eager, model-based learner that constructs an explicit decision boundary during training, while KNN is a lazy, instance-based learner with no explicit training phase.

Decision Boundaries: SVM finds the single maximum-margin hyperplane (or non-linear boundary via kernels) that best separates the classes, based only on the support vectors — the critical boundary points. This produces a smooth, globally optimal, and well-generalizing boundary. KNN's decision boundary is implicit and piecewise, determined locally by whichever K training points happen to be nearest to each query point; it can be highly irregular and jagged, closely following the local density of the training data rather than an optimized global criterion.

Scalability: SVM training complexity scales poorly with the number of training examples (roughly quadratic to cubic in standard formulations), but once trained, prediction is fast since it only depends on the (usually small) set of support vectors. KNN has no training cost at all, but every prediction requires computing distances to all (or many) training points, making inference slow and memory-intensive as the dataset grows — a significant drawback for a large digit-image database queried repeatedly.

Sensitivity to Noise: SVM is relatively robust to noise and outliers because the margin-maximization objective and the (optional) soft-margin parameter C allow some misclassified points without drastically shifting the boundary. KNN is more sensitive to noisy or mislabeled data, especially for small K, since a single noisy neighbour can directly flip a prediction; increasing K mitigates this but can blur genuine class boundaries.

Computational Complexity: SVM: training is the expensive step (solving a quadratic optimization problem), while prediction is $O(\text{number of support vectors})$. KNN: training is $O(1)$ (just storing data), while prediction is $O(n)$ per query in the naive case (n = training set size), or $O(\log n)$ with optimized structures like KD-trees/ball-trees for lower-dimensional data — though these structures degrade in effectiveness for high-dimensional pixel vectors.

Kernel Functions in SVM: For non-linearly separable data — such as digit images where classes are not linearly separable in raw pixel space — SVM applies the kernel trick, implicitly mapping inputs into a

higher-dimensional feature space (via kernels such as polynomial or RBF/Gaussian) where a linear separator can be found, without ever explicitly computing the high-dimensional coordinates. This allows SVM to model complex, curved decision boundaries efficiently while retaining the same margin-maximization guarantees, giving it a significant performance edge over KNN on non-linear, high-dimensional classification problems like digit recognition.

5. Compare K-Means Clustering and Hierarchical Clustering using a dataset with unknown structure. Analyze differences in cluster formation, computational cost, scalability, and interpretability. Discuss how each method handles varying cluster shapes and the impact of choosing the number of clusters.

Overview: Consider a customer-segmentation dataset where the true number and shape of customer groups is unknown in advance. K-Means and Hierarchical Clustering are both unsupervised methods for discovering such structure, but they approach the problem very differently.

Cluster Formation: K-Means is a partitional, centroid-based method: it randomly initializes K centroids and iteratively assigns each point to its nearest centroid, then recomputes centroids, until convergence (minimizing within-cluster variance). Hierarchical Clustering (agglomerative, most commonly) builds a nested tree (dendrogram) of clusters bottom-up, starting with every point as its own cluster and successively merging the closest pair of clusters based on a linkage criterion (single, complete, average, or Ward's), without requiring K to be fixed in advance.

Computational Cost: K-Means is computationally efficient, roughly $O(n \cdot K \cdot i \cdot d)$ where n is the number of points, K the number of clusters, i the number of iterations, and d the dimensionality — this scales well to large customer databases. Hierarchical Clustering is considerably more expensive, typically $O(n^2)$ or $O(n^2 \log n)$ for computing and updating the distance matrix, making it impractical for very large datasets without approximation or sampling.

Scalability: Because of its lower complexity and easy parallelization/mini-batch variants, K-Means scales to millions of records and is the practical choice for large-scale customer data. Hierarchical Clustering does not scale as gracefully; it is typically applied to smaller or moderate-sized datasets, or used after an initial reduction step (e.g., sampling or pre-clustering).

Interpretability: K-Means gives a single flat partition into K groups, which is simple to interpret but offers no insight into sub-group relationships. Hierarchical Clustering produces a dendrogram showing the full nested structure of merges at every similarity level, which is far more interpretable for exploring how clusters relate and for deciding a natural cut-point — useful when analysts want to inspect customer segments at multiple levels of granularity (e.g., broad segments vs. fine-grained niches).

Handling Varying Cluster Shapes: K-Means assumes roughly spherical, similarly-sized clusters because it minimizes Euclidean distance to a centroid; it struggles with elongated, non-convex, or unevenly sized/dense clusters, often splitting or merging them incorrectly. Hierarchical Clustering, especially with single or average linkage, can capture more irregular, non-spherical cluster shapes since it builds structure from pairwise distances rather than assuming a centroid model, though complete-linkage/Ward variants still favour more compact clusters.

Impact of Choosing the Number of Clusters: K-Means requires K to be specified in advance, and results are sensitive to this choice — too few merges distinct groups, too many fragments genuine ones; techniques like the elbow method or silhouette score are used to select K , and poor initialization can also lead to different local optima across runs. Hierarchical Clustering does not require K upfront; instead, the number of clusters is chosen after the fact by cutting the dendrogram at a chosen height/distance threshold, giving more flexibility to explore multiple granularities from a single run rather than having to

re-run the algorithm for each candidate K.

Conclusion: K-Means is the efficient, scalable choice for large datasets with roughly spherical clusters and a reasonably well-estimated K, while Hierarchical Clustering better suits smaller datasets with unknown or irregular cluster structure where interpretability of the full cluster hierarchy is valuable.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand